

Cost-Aware Optimization for Agentic Query Execution

Lunyu Nie

The University of Texas at Austin
lynie@utexas.edu

Yilin Xia

University of Illinois
Urbana-Champaign
yilinx2@illinois.edu

Yiren Liu

University of Illinois
Urbana-Champaign
yiren12@illinois.edu

Christopher Jermaine

Rice University
cmj4@rice.edu

Swarat Chaudhuri

The University of Texas at Austin
swarat@cs.utexas.edu

Abstract

Classical query optimization searches over algebraically equivalent plans that differ only in cost. This assumption breaks once LLM-backed operators enter the picture: their placement, ordering, and granularity jointly determine both dollar cost and answer quality, and the right choice among the alternatives is often revealed only at runtime. We formalize this setting as **Agentic Query Execution**, a query execution paradigm in which agent-based planning is interleaved with execution, and **agent workflow optimization** becomes the analogue of classical query optimization. We then present **EnumGRPO**, a self-improving optimizer for this setting. During a learning stage, EnumGRPO enumerates query plans over decisions such as execution paradigm, operator type, operator placement, selectivity scope, and projection width, then distills quality-cost feedback into reusable planning heuristics via in-context reinforcement learning. Across four databases in SWAN, EnumGRPO achieves 35.4% execution accuracy at \$0.011 per query in LLM-operator cost, a 311× cost reduction over the hybrid query baseline.

PVLDB Reference Format:

Lunyu Nie, Yilin Xia, Yiren Liu, Christopher Jermaine, and Swarat Chaudhuri. Cost-Aware Optimization for Agentic Query Execution. PVLDB, 14(1): XXX-XXX, 2020.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at URL_TO_YOUR_ARTIFACTS.

1 Introduction

Classically, query optimization applies algebraic transformations without changing the answer by design: one can push a selection past a join, reorder associative operators, or choose between index scans and hash joins, and the result is guaranteed to be equivalent. The optimizer’s job is to navigate the space of these equivalent plans and pick the most efficient one [5, 11, 30].

This narrow view, that all candidate plans produce the same answer and differ only in cost, breaks down once LLM-backed operators enter the picture. A recent line of hybrid database systems

[10, 12, 15, 21, 29, 38] has extended SQL with semantic operators that invoke LLMs to normalize values, resolve ambiguities, and fill knowledge gaps that the schema alone cannot supply. Unlike a hash join or an index scan, however, an LLM operator is *approximate*: it can produce different answers depending on how much context it receives, which rows it processes, and where in the plan it is placed. At the same time, each LLM invocation is orders of magnitude more expensive than a relational operator. The result is that structural decisions about the workflow, the very choices that a classical optimizer treats as cost-neutral equivalences, now jointly determine both answer quality and dollar cost. Finding the best plan is no longer a search over equivalent alternatives; it is a search over plans that differ in both cost *and* correctness.

For example, consider the query in Figure 1: “Who are the 8 youngest Japanese F1 drivers, and how old are they?” The stored table lacks both *nationality* and *date-of-birth*, and an LLM can supply the missing knowledge. But there are structurally different ways to do so. Workflow A calls the LLM on every driver to infer nationality, then filters, then calls it again for birth dates. Workflow B first applies a cheap SQL heuristic to narrow candidates, then batches a single LLM call over the survivors. In this example, both options return the correct answer—though this cannot be guaranteed by looking at the queries themselves—while the latter costs 20× less. The question is: how do we optimize for both cost and accuracy when LLM operators are integrated into the data analytics pipeline?

This requires an optimizer whose decisions are made as part of query execution. Existing hybrid systems [10, 15, 21, 29] separate optimization from execution: the plan is fixed before any tuple is processed and never revised against intermediate results. But for many analytical workloads, the need for semantic reasoning emerges from query execution itself. The decisive choices depend on cardinalities, value distributions, and schema and data semantics that only become observable at runtime. Intermediate results determine not only *whether* an LLM operator is needed, but also *which* operator form is appropriate, *where* in the plan it should appear, *how selectively* it should be applied, and *how much context* it should consume.

The unit of execution is therefore no longer a pre-specified hybrid query but an *agent workflow* that we call **Agentic Query Execution**. Rather than committing to a fully specified SQL+LLM plan, the system executes one step, observes the resulting relation, and decides the next action: which operator to invoke, on which intermediate relation, and at what granularity. SQL remains responsible for deterministic relational computation, while LLM operators are invoked only when intermediate results indicate that semantic

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.

Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

reasoning is required. We use Agentic Query Execution as the execution setting, but the core systems problem is **agent workflow optimization**: better runtime decision-making over dynamically constructed workflows. The optimizer must decide how to decompose a task into steps, when to push narrowing operations into SQL before invoking an LLM, and how much data each LLM call should inspect, while taking both accuracy and cost into account. This connects our setting to adaptive query processing and learned query optimization [3, 23, 34, 36, 39].

To address this, we introduce **EnumGRPO**, a self-improving optimizer for agent workflow optimization. During a learning stage, EnumGRPO explicitly enumerates diverse candidate workflows for each query over five orthogonal planning axes (execution paradigm, operator type, operator placement, selectivity scope, and projection width). Each axis has a classical analogue, such as predicate push-down, late materialization, or access-path selection [1, 30], but now governs both cost and answer quality, not cost alone. Candidates are executed and scored by a joint quality-cost objective. Group-relative advantages isolate the effect of structural decisions from query-level difficulty, and a four-stage experience distillation pipeline converts contrastive feedback into reusable planning heuristics. At query time, the learned policy applies these cross-query heuristics without runtime enumeration to navigate the cost-quality Pareto frontier of agentic workflows.

The entire procedure operates through *in-context reinforcement learning*: policy improvement flows through the agent’s prompt, without gradient updates to model weights. We adopt this design for two reasons. First, the learned heuristics are encoded as natural language and remain human-readable and auditable. Second, the policy is portable: because improvement lives in the prompt rather than the weights, it is decoupled from any specific backbone. This is essential for a long-lived DBMS, whose agent model is upgraded repeatedly and is increasingly a closed-source frontier model served through an API that cannot be fine-tuned. A weight-tuned policy would need re-training on every upgrade, if it could be trained at all, whereas our natural-language heuristics carry across model generations intact.

We evaluate on four databases in SWAN [38], whose tasks require semantic reasoning and knowledge beyond what is explicitly stored in the database. EnumGRPO achieves **35.4%** execution accuracy, compared with **19.2%** for Agentic Text2SQL and **30.0%** for Agentic BlendSQL. The learned optimizer simultaneously reduces LLM-operator cost to \$0.011 per query, a 311× reduction compared to the hybrid query baseline (\$3.42) and a 72% reduction over the unoptimized agentic system (\$0.039), while LLM-operator token consumption falls by 64%.

In summary, this paper makes the following contributions:

- We formulate **Agentic Query Execution** and its query optimization as an **agent workflow optimization** problem, in which agent-based planning is interleaved with execution over intermediate results. It generalizes classical query optimization from cost-only search to joint cost-quality search over dynamically constructed plans.
- We characterize its plan space along five orthogonal axes: execution paradigm, operator type, operator placement, selectivity scope, and projection width.

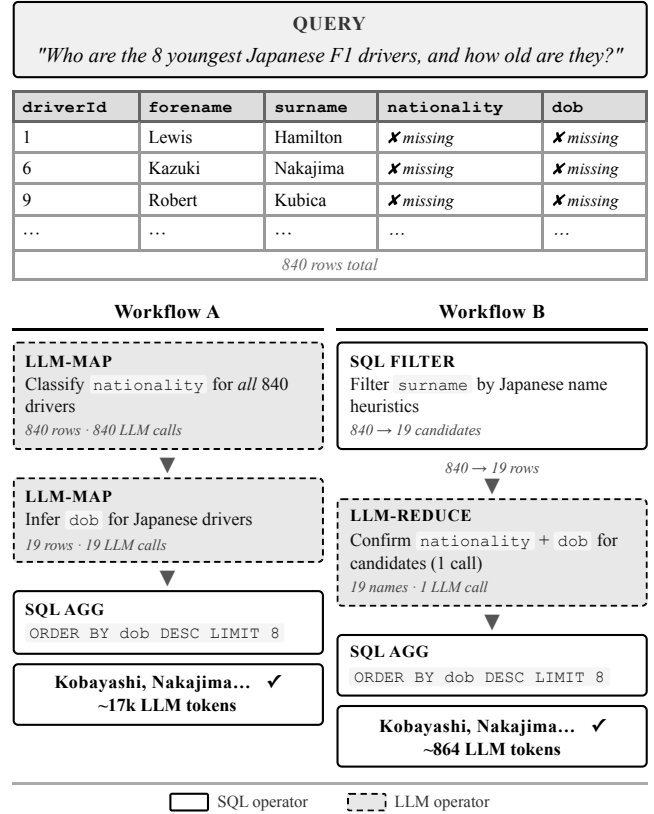


Figure 1: Two valid workflows for the same query over an F1 drivers table with missing nationality and dob columns. Both return the correct answer, but Workflow B pushes a SQL filter before the LLM and batches inference into a single call, reducing token cost by 20×.

- We propose **EnumGRPO**, a learned optimizer that uses structured plan enumeration during a learning stage paired with in-context reinforcement learning to distill reusable query optimization heuristics, enabling better workflow construction at query time without model retraining.
- We demonstrate on four databases in **SWAN** that our optimized system achieves 35.4% execution accuracy at \$0.011 per query in LLM-operator cost, simultaneously improving answer quality and reducing LLM-operator cost by 311× compared to the hybrid query baseline.

2 Problem Formulation

We first formalize **Agentic Query Execution** as a query execution paradigm in which agent-based planning is interleaved with execution over an extended relational algebra: after each step, the system observes new intermediate results and decides which operator to apply next, whether to stay inside the DBMS, and when to pay for approximate LLM-backed semantic processing.

2.1 Task Setting

Let $\mathcal{D}$ denote a relational database instance with base relations $\{R_i\}_{i=1}^m$ and schemas $\{\text{sch}(R_i)\}$. An analytics task is specified by a natural-language query q , optionally with additional textual context such as column descriptions or domain-specific conventions. The system is equipped with a toolset:

$$K = K_{\text{SQL}} \cup K_{\text{LLM}} \cup K_{\text{AUX}},$$

where K_{SQL} contains DBMS-resident relational processing, K_{LLM} contains LLM-backed semantic operators, and K_{AUX} contains supporting primitives such as schema inspection, sampling, or summarization.

Execution proceeds for up to T turns. Before turn t , the system maintains a *workspace* W^t containing named intermediate relations, schema and catalog fragments, lightweight statistics or samples, and the execution trace accumulated so far. Let y denote the final output produced when execution terminates.

2.2 Logical Operators, Physical Tools, and Actions

Logical operators. Mirroring the logical-to-physical separation in classical query processing [11], we distinguish logical operators from their physical tool implementations. A logical operator is a typed transformation over workspace artifacts. This includes relational operators expressible in SQL, such as selection, projection, join, grouping, and aggregation, as well as semantic operators whose behavior is realized through LLM inference [16, 28, 29].

Let $\mathcal{R}$ denote the set of all possible finite relations, let $\mathcal{R}_{\text{SCORE}}$ denote relations augmented with a real-valued score attribute, and let $\mathbf{1}$ denote singleton outputs. Representative semantic operators include:

$$\begin{aligned} \text{llm_map} &: \mathcal{R} \rightarrow \mathcal{R} \text{ (transformation)} \\ \text{llm_filter} &: \mathcal{R} \rightarrow \mathcal{R} \text{ (selection)} \\ \text{llm_rank} &: \mathcal{R} \rightarrow \mathcal{R}_{\text{SCORE}} \\ \text{llm_reduce} &: \mathcal{R} \rightarrow \mathbf{1} \\ \text{llm_join} &: \mathcal{R} \times \mathcal{R} \rightarrow \mathcal{R}. \end{aligned}$$

Unlike their relational counterparts, semantic operators are *approximate*: the quality of their output depends on the cardinality and content of the input relation, so placing the same logical operator at different points in a workflow can yield different-quality results.

Physical tools. A tool $k \in K$ is a callable physical implementation of one or more logical operators. A SQL tool may compile directly to DBMS execution, while an LLM tool fixes a model family, serving configuration, prompt template, batching policy, and any surrounding SQL pre- or post-processing. A tool may implement a single operator or a macro that expands into a short operator pipeline.

Actions. An action is one concrete invocation

$$a_t = (k_t, \eta_t, w_t),$$

where $k_t \in K$ is the chosen tool, η_t are its parameters, and $w_t \subseteq W^t$ identifies the workspace artifacts consumed by the action. Executing a_t produces updated artifacts and yields a new workspace W^{t+1} . SQL actions are deterministic for a fixed database snapshot, while LLM-tool actions can be stochastic and approximate.

2.3 Agentic Workflow Optimization

Within this new execution paradigm, query optimization becomes *agentic workflow optimization*. Unlike classical optimization over equivalent plans, the optimizer must now navigate plans that differ in both cost and answer quality.

State and feasible actions. At query time, the optimizer observes only the current task and workspace, without any reference answer. We summarize the decision-relevant context at turn t as a state

$$s_t = \Sigma(q, W^t),$$

where $\Sigma(\cdot)$ is a lossy compression function that selects what agent observes from the workspace. Let $\mathcal{S}$ denote the set of reachable states. Given $s_t \in \mathcal{S}$, the optimizer selects from a feasible action set $\mathcal{A}(s_t)$, containing all tool invocations whose required inputs are present in W^t , together with a distinguished action `STOP` that terminates execution and returns output y from the current workspace.

Workflow cost. Each non-terminal action incurs a nonnegative cost $c(s_t, a_t)$, which may reflect latency, computation overhead, or monetary expense, depending on the deployment setting. Since a tool can expand into multiple physical substeps, we interpret

$$c(s_t, a_t) = c_{\text{SQL}}(s_t, a_t) + c_{\text{LLM}}(s_t, a_t) + c_{\text{AUX}}(s_t, a_t),$$

where the three terms capture DBMS work, LLM usage, and auxiliary processing, respectively. In practice, c_{LLM} dominates the other two by orders of magnitude.

Workflow utility. Let $U(y; q, \mathcal{D}) \in \mathbb{R}$ denote the utility of the final output y , measuring answer quality for the user task. We leave U abstract: it may correspond to exact-match correctness, a soft similarity score, or any task-specific quality measure. Crucially, U is not directly observable at query time because no reference answer is available, so the optimizer must trade estimated quality against resource consumption.

Policy and objective. A policy $\pi(a \mid s)$ defines a distribution over feasible next actions given the current state. Executing π induces an adaptive workflow

$$\tau = (a_1, a_2, \dots, a_T, \text{STOP}),$$

whose later actions depend on intermediate relations materialized during earlier turns. Each workflow produces a trajectory reward that trades output quality against cumulative cost:

$$G(\tau) = U(y; q, \mathcal{D}) - \lambda \sum_{t=1}^T c(s_t, a_t), \quad (1)$$

for a tradeoff parameter $\lambda \geq 0$. The primary optimization problem is to maximize the expected reward under policy π :

$$\max_{\pi} J(\pi) = \max_{\pi} \mathbb{E}_{\tau \sim \pi} [G(\tau)]. \quad (2)$$

Concretely, the policy must make structural planning decisions at each step, such as which operator to invoke and where to place it, while accounting for the fact that LLM-backed operators are both expensive and approximate.

2.4 Learned Optimization via In-context Reinforcement Learning

The optimization problem above is inherently sequential: each planning decision changes the workspace and thus the set of useful downstream actions. Improving the planner therefore requires credit assignment over entire workflow trajectories rather than supervision on a single next action, which naturally suggests a reinforcement-learning view. The workflow forms a finite-horizon Markov Decision Process (MDP):

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, \rho_0, T),$$

where $P(s_{t+1} | s_t, a_t)$ is the transition kernel induced by executing a_t and updating the workspace, ρ_0 is the initial state determined by the database instance and the task query, and T is the horizon length.

During the offline learning stage, we have access to tasks with reference outputs y^* , which lets us instantiate the abstract utility U with a concrete benchmark-specific score $S(\hat{y}, y^*; q, \mathcal{D})$ that compares the predicted output to the reference. The trajectory reward from Eq. (1) becomes

$$G(\tau) = S(\hat{y}, y^*; q, \mathcal{D}) - \lambda \sum_{t=1}^T c(s_t, a_t),$$

where the tradeoff parameter λ may differ between the learning stage and query time. Maximizing $J(\pi)$ over this reward would normally require updating model parameters.

However, updating an LLM’s weights through gradient-based training is expensive, may degrade general reasoning capabilities, and is infeasible when the model is accessed only through an API. We therefore adopt *in-context reinforcement learning* (ICRL): the model parameters θ remain fixed, and policy improvement operates through the prompt context supplied to the model [2, 6, 32].

Let $\mathcal{C}$ denote a ground set of admissible context elements such as planning guidelines, demonstrations, or instructions. A prompt context $c \subseteq \mathcal{C}$ is a selection of these elements injected into the model’s system prompt. The induced policy for a given context c is $\pi_{\theta,c}(a_t | s_t)$, where c collectively conditions how the frozen LLM maps the current state to the next action. The ICRL objective restricts the optimization to the space of context subsets:

$$\max_{c \subseteq \mathcal{C}} J(\pi_{\theta,c}) = \max_{c \subseteq \mathcal{C}} \mathbb{E}_{\tau \sim \pi_{\theta,c}} [G(\tau)]. \quad (3)$$

This is policy optimization over a restricted search space, avoiding the substantial costs of gradient-based parameter updates. Note that Eq. (3) is solved offline during the learning stage, where oracle outputs y^* provide the concrete score S needed to evaluate trajectories. At query time, the system applies the learned context c to construct workflows without access to ground-truth answers. Section 3 instantiates this objective with our specific algorithm.

3 Method

Optimizing Agentic Query Execution workflows requires answering two questions: *what* are the structural decisions that govern the quality-cost tradeoff of an execution plan, and *how* should the system learn to make them well? Section 3.1 addresses the first question by identifying five orthogonal planning dimensions, each grounded in a classical query-optimization principle. Section 3.2

then presents **EnumGRPO**, a self-improving optimizer that enumerates over this plan space during its learning stage and distills the results into reusable planning heuristics without gradient updates.

3.1 Plan Space of Agentic Query Execution

A classical query optimizer explores a structured space of physical execution plans, selecting among join algorithms, operator orderings, access paths, and materialization strategies to minimize estimated cost while preserving answer equivalence [30]. When a workflow interleaves SQL operators and LLM-backed semantic operators, the planner faces analogous structural decisions that jointly determine the quality-cost tradeoff. We identify five orthogonal axes that span the principal planning decisions in this setting, summarized in Table 1. We discuss each below and illustrate the two choices on a running example.

Example 1. A table `Companies` stores company records but has no sector column:

id	name	revenue	headquarters
1	Medtronic	31,200M	Minneapolis, MN
2	Genesis	8,400M	Dearborn, MI
3	Stryker	19,800M	Kalamazoo, MI
4	Apex Solar	2,100M	Austin, TX
5	...	...	...

A user asks: “*What is the total revenue of healthcare companies?*” Answering this query requires the system to infer each company’s sector from its name and other attributes using world knowledge not present in the database.

Execution paradigm. In compiled query execution, the system generates a specialized machine-code path for each query rather than interpreting tuples through generic iterator operators, collapsing per-tuple overhead by orders of magnitude [27]. The same compile-vs.-interpret choice arises when an agentic workflow requires LLM inference. In the *data-driven* paradigm, tuples are routed through an LLM operator that performs semantic inference per invocation, incurring $O(|R|)$ LLM calls. In the *code-driven* paradigm, the LLM inspects a small sample and synthesizes a deterministic SQL rule (a CASE WHEN expression, a regular-expression replacement, or a UDF) that the DBMS executes natively over all rows, incurring only $O(1)$ LLM calls. The tradeoff is that rule synthesis works well when the mapping from existing attributes to the derived value follows a detectable pattern, but fails when the semantic reasoning required is too heterogeneous to capture in a single rule; either way, the $O(|R|)$ -to- $O(1)$ gap makes execution paradigm the single largest lever on token cost.

In Example 1:

DATA-DRIVEN The LLM classifies each company’s sector individually, issuing one call per row.

CODE-DRIVEN The LLM synthesizes a CASE WHEN mapping well-known company names to sectors and executes it as pure SQL.

Operator type. A query optimizer chooses among join algorithms with different per-tuple and per-relation cost profiles: a nested-loop join processes one tuple at a time from the inner relation, while a hash join builds a summary structure over one input and probes it with the other [26]. The same row-level vs. set-level choice

Table 1: The plan enumeration space for Agentic Query Execution. Each axis represents a structural planning decision with a direct analogue in classical query optimization.

Axis	Values	Description	Optimizer analogue
Execution paradigm	DATA-DRIVEN / CODE-DRIVEN	Choose between per-tuple LLM execution and one-shot rule synthesis.	JIT query compilation [27]
Operator type	SCALAR / AGGREGATE	Choose whether the LLM processes rows individually or in aggregate.	Join algorithm selection [26]
Operator placement	PRE- / POST-AGGREGATION	Choose whether the LLM runs before or after SQL aggregation.	Predicate pushdown [30]
Selectivity scope	FULL / TARGETED	Control how much of the filtered working set reaches the LLM.	Access-path & materialization [25, 30]
Projection width	NARROW / WIDE	Control how much row context is exposed to the LLM.	Late materialization [1]

applies to LLM operators. A *scalar* operator, e.g., `llm_map`, applies the LLM to each row independently, producing a per-row result (an inferred attribute, a boolean label, a normalized value). An *aggregate* operator, e.g., `llm_reduce`, passes an entire relation or a grouped partition to the LLM in a single call, producing an aggregate or summary. Scalar processing offers fine-grained control and is robust when rows require different reasoning, while aggregate processing amortizes prompt overhead and is more effective when the LLM can reason about the full context simultaneously, for example when cross-referencing related entries, resolving ties, ranking candidates, or detecting duplicates.

In Example 1:

SCALAR Classifies each company’s sector one row at a time.
AGGREGATE Receives all company names at once and returns only those in healthcare, cross-referencing entries to recognize that “Medtronic” and “Stryker” both belong to the same sector.

Operator placement. Predicate pushdown is a cornerstone of relational optimization: the optimizer pushes selective predicates as early as possible in the plan tree, but it cannot push a predicate below the operator that produces the column it references [30]. The same placement constraint governs LLM operators in agentic workflows, with the additional complication that the “selectivity” of an LLM operator is unknown a priori and may only become apparent after inspecting intermediate results. *Pre-aggregation* placement positions the LLM operator mid-pipeline, before downstream SQL operators that depend on the semantically resolved value. This is necessary when an LLM-derived column appears on a join key or in a WHERE predicate, because the SQL operator cannot evaluate correctly until the attribute has been inferred. *Post-aggregation* placement defers the LLM to the end of the pipeline, after all SQL has been applied, so it operates on a smaller surviving result set.

In Example 1:

PRE-AGGREGATION The `GROUP BY sector` requires the inferred sector column, so the LLM must classify companies before the aggregation can run.
POST-AGGREGATION If a variant query asked for the total revenue of a specific company by name (no sector dependency), the LLM could run after SQL has already filtered the result set.

Selectivity scope. Access-path selection determines how much of a relation the engine actually reads: an index-restricted scan touches only qualifying rows, while a full scan reads the entire table [30]. Materialization boundaries further control which intermediate results are physically produced and which are pipelined [25]. In agentic workflows, the analogous decision is how much of the

working set reaches the LLM. Under *full* scope, the entire filtered relation is passed to the LLM operator. Under *targeted* scope, the planner identifies the minimum set of rows strictly required to answer the query and restricts the LLM’s input with a tight SQL predicate.

In Example 1:

FULL Every company is sent to the LLM for sector classification.
TARGETED A SQL heuristic such as `WHERE name ILIKE ‘%medical%’ OR name ILIKE ‘%health%’` resolves easy cases without the LLM; only the remaining ambiguous rows are classified.

Projection width. Column stores defer tuple reconstruction until after all predicates have been evaluated, avoiding the cost of materializing columns that are never needed [1]. This late-materialization principle extends directly to LLM operator inputs. Under *narrow* projection, only the target column and the primary key are included in each LLM call, minimizing per-call token cost. Under *wide* projection, the full row or a semantically relevant subset of columns is included, giving the LLM the surrounding context it may need to make accurate inferences. The tradeoff is strictly between token cost and inference accuracy, and it is orthogonal to the other four axes.

In Example 1:

NARROW Only name and the primary key are sent; token cost is minimal.
WIDE Including headquarters lets the LLM disambiguate: “Genesis” in “Dearborn, MI” is likely automotive, not healthcare.

Contrasting workflows. The compositional power of the plan space becomes visible when different axis combinations are applied to Example 1. Consider three workflows:

- (1) **Conservative:** DATA-DRIVEN, SCALAR, PRE-AGGREGATION, FULL scope, WIDE projection. The agent infers the sector of every company row-by-row, including headquarters for context, then runs `GROUP BY sector` and `SUM(revenue)`. This yields the highest accuracy but incurs $O(|R|)$ LLM calls with wide prompts, making it the most expensive option.
- (2) **Aggressive:** CODE-DRIVEN, PRE-AGGREGATION, FULL scope, NARROW projection. The agent samples a handful of rows, asks the LLM to synthesize a `CASE WHEN` mapping company names to sectors, and runs the resulting SQL rule natively. Total cost is $O(1)$ LLM calls, but accuracy degrades for companies the rule cannot cover.

- (3) **Balanced:** DATA-DRIVEN, AGGREGATE, PRE-AGGREGATION, TARGETED scope, NARROW projection. The agent first narrows the table with a SQL filter (e.g., revenue above a threshold), passes the surviving company names to an aggregate LLM operator in a single batch call, and aggregates over the returned labels. Cost scales with the filtered set rather than the full table, and set-level reasoning lets the LLM cross-reference entries.

All three plans are valid for the same query; which one dominates depends on the data distribution, table size, and the user’s quality-cost preference. The optimizer’s job is to learn which regions of this space tend to perform well for different query and data characteristics.

Adaptive planning. Because Agentic Query Execution interleaves planning with execution, the agent does not commit to axis values blindly. It profiles intermediate results, sampling rows and inspecting value distributions, before selecting its strategy. This mirrors the Eddies principle [3] of deferring routing decisions to runtime based on observed selectivities. The learned heuristics in ε guide which axis values to prefer given observed data characteristics.

From plan space to learned optimization. Executing different axis combinations on the same query produces workflows with different quality and cost. The central challenge is *credit assignment*: determining which structural choices caused the observed differences and distilling these lessons into heuristics that generalize across queries. This shifts the optimization challenge from the enumeration paradigm of System R [30] toward the learned-optimizer paradigm of systems like Bao [23], which learn value models over coarse structural hints rather than enumerating fine-grained physical plans. EnumGRPO, described next, combines both: it enumerates over the axis space during learning to generate contrastive signal, then distills cross-query heuristics that replace enumeration at query time.

3.2 Learning to Navigate the Plan Space

The plan space above defines what an optimizer must decide; this section describes how the system learns to decide well. Section 2 cast the problem as in-context reinforcement learning (ICRL), where the LLM’s weights are frozen and all policy improvement flows through a prompt context c . To solve this ICRL objective we need a mechanism that (i) generates diverse candidate behaviors for the same query, (ii) scores them, and (iii) distills the relative outcomes into reusable guidance.

GRPO. Group Relative Policy Optimization (GRPO) [31] provides exactly this structure. For each input, GRPO samples a group of N rollouts, scores them, and normalizes rewards within the group so that above-average attempts receive positive advantage and below-average ones receive negative advantage. This group-relative signal fits the Agentic Query Execution setting particularly well. A healthcare-revenue query over a 10-row table and a sector-classification query over 10,000 rows produce reward distributions on entirely different scales; within-group normalization absorbs this variance and yields a stable learning signal without global reward calibration. More importantly, because each rollout group targets the

same query, the contrastive comparison isolates the effect of plan-axis choices and aligns naturally with the credit-assignment problem identified in Section 3.1: if a CODE_DRIVEN/PRE_AGG workflow scores higher than a DATA_DRIVEN/POST_AGG alternative on the same data, the advantage directly reflects the structural decision rather than query-level difficulty.

From GRPO to EnumGRPO. Standard GRPO relies on temperature sampling to produce diverse rollouts. **EnumGRPO** replaces this undirected diversity with structured plan enumeration, seeding each rollout in a group with a distinct axis-value assignment from the plan space. This guarantees that rollout groups cover meaningfully different structural alternatives, producing stronger contrastive signal for credit assignment. EnumGRPO instantiates the ICRL objective from Eq. (3) without any gradient updates: the LLM planner’s weights remain fixed, and policy improvement operates entirely through an evolving experience pool $\varepsilon \subseteq C$ that is prepended to the agent’s system prompt.

The algorithm has three components. First, a *plan enumerator* exploits the structured plan space from Section 3.1 to generate diverse candidate workflows per query, guaranteeing that rollout groups cover different regions of the axis space rather than collapsing under temperature noise alone. Second, a *multi-objective reward* scores each workflow on both answer quality and LLM-operator cost, instantiating the trajectory reward $G(\tau)$ from the formulation with quality signals at multiple granularities to avoid reward sparsity. Third, a *four-stage experience distillation pipeline* converts group-relative advantage signals into reusable planning heuristics that update ε across learning batches.

3.2.1 Plan Enumeration To seed each rollout group with diverse strategies, the PlanEnumerator selects N axis-value assignments before execution begins. Given the query, the database schema, and a small row sample, a single LLM call proposes N assignments as structured JSON, each specifying a concrete choice along every axis. A diversity constraint ensures the final set spans the most consequential structural alternatives rather than clustering in one region of the plan space. Each assignment is then injected into the agent prompt as a soft recommendation that the agent may override when the data warrants a different approach.

3.2.2 Multi-Objective Reward Each rollout receives a composite reward combining answer quality at three granularities with a cost penalty. A binary exact-match score alone is too sparse: for harder queries, most rollouts score zero, collapsing advantages and making near-correct workflows indistinguishable from completely wrong ones. We therefore decompose quality into result-level, tuple-level, and cell-level components that credit partially correct outputs.

Execution Accuracy (EX). Let $\hat{R}$ denote the agent’s output relation and R^* the ground-truth relation. The coarsest signal is exact-match correctness [9, 13, 17]:

$$\text{EX}(\hat{R}) = 1[\hat{R} = R^*]. \quad (4)$$

EX provides an unambiguous task-completion signal but, used alone, would yield a reward of zero for any rollout that retrieves most rows correctly yet misses or misformats a single value.

Tuple-level F1 (TupleF1). To reward partial overlap at the record level, we use a tuple-level F1 score drawn from entity resolution

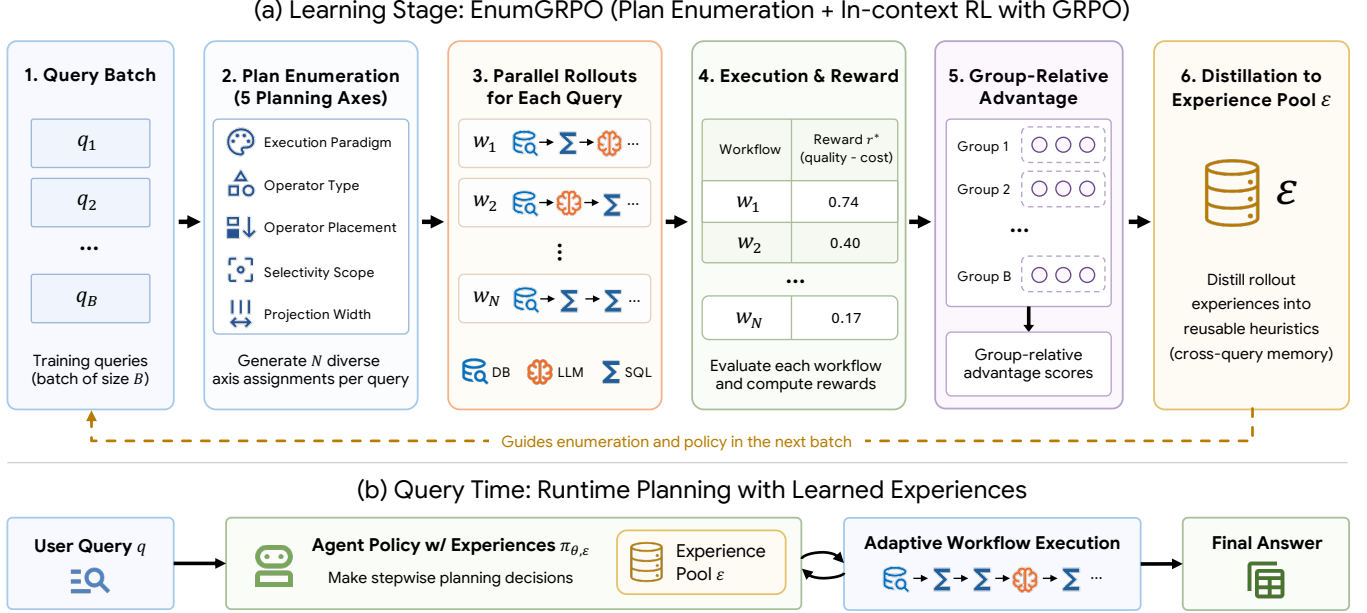


Figure 2: Overview of EnumGRPO. (a) During the learning stage, queries are processed in batches: a PlanEnumerator generates N diverse axis assignments per query, each seeded workflow is executed in parallel and scored by a composite reward to compute group-relative advantages, then distilled into reusable heuristics stored in the experience pool ε . (b) At query time, the agent applies the learned ε to make stepwise planning decisions and executes an adaptive workflow without enumeration.

evaluation [14, 19, 24]:

$$\text{TupleF1}(\hat{R}) = \frac{2|\hat{R} \cap R^*|}{|\hat{R}| + |R^*|}, \quad (5)$$

where $\hat{R} \cap R^*$ counts tuples matched by primary key and all attribute values. TupleF1 gives credit when the agent retrieves a correct subset or superset of tuples, producing a nonzero reward even when the result set is not an exact match.

Cell-level F1 (CellF1). The finest-grained signal operates at the individual cell level [7, 28, 33]:

$$\text{CellF1}(\hat{R}) = \frac{2 \sum_c |\hat{R}_c \cap R_c^*|}{\sum_c (|\hat{R}_c| + |R_c^*|)}, \quad (6)$$

where $\hat{R}_c$ and R_c^* are the multisets of values in column c . CellF1 surfaces differences between rollouts that are invisible at the tuple level, for instance when two workflows return the same rows but one infers a cell value correctly while the other does not.

Composite reward. The three granularities are combined into a single quality score for each rollout producing output $\hat{R}$ to query q :

$$r = w_{\text{EX}} \cdot \text{EX}(\hat{R}) + w_{\text{TUPLE}} \cdot \text{TupleF1}(\hat{R}) + w_{\text{CELL}} \cdot \text{CellF1}(\hat{R})$$

Since queries of different complexity produce token counts on different scales, we normalize cost within each rollout group. For N rollouts on the same query, let t_i be the token count and s_i the number of tool calls in rollout i . The composite within-group cost rank is

$$\rho_i = w_{\text{TOKEN}} \cdot \frac{t_i - \min_j t_j}{\max_j t_j - \min_j t_j} + w_{\text{STEP}} \cdot \frac{s_i - \min_j s_j}{\max_j s_j - \min_j s_j}.$$

The final reward is $r_i^* = r_i - w_{\text{COST}} \cdot \rho_i$, making the cost signal invariant to absolute token scale across queries of different difficulty. The weight w_{COST} is kept small relative to the quality weights so that cost acts as a tiebreaker among rollouts with similar correctness.

3.2.3 Group-Relative Advantage Following GRPO [4, 31], advantages are computed group-relatively for each query. For N rollouts with rewards $r_1^*, \dots, r_N^*$:

$$A_i = \frac{r_i^* - \mu}{\sigma + \delta}, \quad \mu = \frac{1}{N} \sum_j r_j^*, \quad \sigma = \sqrt{\frac{1}{N} \sum_j (r_j^* - \mu)^2}.$$

We additionally decompose the advantage into utility and cost components, A_i^{UTIL} and A_i^{COST} , by applying the same z-score normalization to quality scores and negated cost ranks independently. These decomposed signals are surfaced during distillation to help attribute outcomes to specific plan-axis choices.

3.2.4 Experience Distillation The core policy-improvement mechanism is a four-stage LLM pipeline that converts group-relative advantages into reusable planning heuristics. No gradient steps are taken: the “update” to the policy is a textual edit to the persistent experience pool ε that conditions all subsequent rollouts. The four stages form a pipeline from per-rollout observation to global pool update:

Stage 1: Trajectory summarization. For each rollout, an LLM compresses the execution trajectory into a compact behavioral description. The summary receives the query, output, decomposed advantages ($A_i, A_i^{\text{UTIL}}, A_i^{\text{COST}}$), and a per-step execution profile that records tool arguments, row cardinality, and LLM token breakdown.

Algorithm 1 EnumGRPO Learning Loop

Require: database $\mathcal{D}$, query set $\mathcal{Q}$, enumeration axes $\mathcal{L}$

```

1:  $\varepsilon \leftarrow \emptyset$  ▷ init experience pool
2: for each batch  $\{q_1, \dots, q_B\} \subseteq \mathcal{Q}$  do
3:   for each query  $q_i$  do
4:      $\{\phi_j\}_{j=1}^N \leftarrow \text{PlanEnumeration}(q_i, \mathcal{D}, \mathcal{L}, N)$ 
▷ enumerate N diverse axis assignments
5:     for  $j = 1$  to  $N$  in parallel do
6:        $\tau_{ij} \leftarrow \text{Rollout}(q_i, \phi_j, \varepsilon)$  ▷ execute with hint  $\phi_j$ 
7:        $r_{ij} \leftarrow \text{Reward}(\tau_{ij})$  ▷ multi-objective score
8:     end for
9:     Compute group-relative advantages  $A_{ij}$  from  $r_{i1}, \dots, r_{in}$ 
10:   end for
11:    $\varepsilon \leftarrow \text{Distill}(\{(\tau_{ij}, A_{ij})\}, \varepsilon)$  ▷ experience distillation
12: end for
13: return experience pool  $\varepsilon$ 

```

This profile supplies a dense, step-level signal that complements the end-to-end reward: it makes localized inefficiencies visible, such as a large `llm_map` input that could have been pre-filtered by SQL, even when the final answer happens to be correct.

Stage 2: Contrastive analysis. For each query, all rollout summaries are presented together, annotated with their advantages and plan-axis labels. The LLM contrasts high-advantage and low-advantage attempts and proposes generalizable experiences: short, action-oriented guidelines such as “apply SQL narrowing before `llm_map` when the inferred column is not on the join key.” The axis labels allow the LLM to reason directly about which structural choice drove the outcome.

Stage 3: Pool update. Proposed experiences are compared against the current pool. An LLM classifies each proposal as genuinely new (ADD), a refinement of an existing entry (UPDATE), or a contradiction (DELETE).

Stage 4: Batch consolidation. All per-query operations are merged in a single consolidation call that deduplicates semantically overlapping additions and ensures global consistency. The consolidated pool is written to disk and appended to the base agent prompt, producing the updated ε that parameterizes the policy $\pi_{\theta, \varepsilon}$ in subsequent rollouts.

3.3 Learning and Execution

Algorithm 1 gives the full EnumGRPO learning loop. Queries are processed in batches of B . For each batch, N rollouts are run per query in parallel, each seeded with a different plan-axis assignment from the enumerator. Rewards are computed, advantages are derived group-relatively, and the four-stage pipeline updates the experience pool. The updated pool is injected into the agent prompt for the next batch.

At query time, the learned experience pool ε is prepended to the agent’s system prompt. No plan enumeration is performed: the agent receives a new query and its database, plans and executes a workflow step by step using the heuristics in ε , and returns a final answer. The learned experiences thus serve as a lightweight planning model that guides the agent’s axis-level decisions without any runtime overhead beyond the additional prompt tokens.

Table 2: Statistics of databases in SWAN [38].

Database	Tables	Rows/Table	Cols Dropped
California Schools	3	3,870	14
European Football	8	23,209	12
Formula One	14	36,736	13
Superhero	10	1,056	9

4 Experimental Setup

4.1 Evaluation

Benchmark. Our evaluation requires a benchmark whose queries cannot be answered by relational operators alone but instead demand semantic reasoning or knowledge absent from the schema, while still providing deterministic ground truth for automated scoring. We evaluate on SWAN [38], a cross-domain benchmark of 120 beyond-database questions over four real-world SQLite databases derived from Bird [18]. Each question has partial information grounded in the relational tables but additionally requires world knowledge that is absent from the schema (e.g., mapping a superhero name to its publisher, or a zip code to a city). SWAN creates this setting by removing selected columns and join paths from the complete Bird databases; the system under test sees only the incomplete schema, while ground-truth answers are defined by executing gold SQL on the original databases. We additionally drop columns whose values are trivially recoverable via a single join, ensuring that no purely relational path to the answer remains and every query genuinely requires LLM-backed inference. Table 2 summarizes the resulting database statistics. We split the 120 questions randomly into 40 for EnumGRPO learning and 80 for evaluation.

Metrics. We measure result quality at different granularities with Execution Accuracy (Eq. 4), Tuple-level F1 (Eq. 5), and Cell-level F1 (Eq. 6), defined in Section 3.2.2. We also report the workflow efficiency in terms of wall-clock latency and LLM-operator costs.

4.2 Implementation Details

Testbed settings. All experiments run on an Ubuntu 22.04 server equipped with two Intel Xeon Platinum 8352Y CPUs (64 cores / 128 threads) and 504 GB RAM. No GPU is required because all LLM inference is served through cloud API endpoints. We use DuckDB v1.5.0 as the query execution engine, exposed to the agent via a Model Context Protocol (MCP) stdio server.

Agent settings. The agent is built on the OpenHands SDK v1.11.5. All systems share the same LLM backbone so that performance differences reflect only planning strategy. Agent planning uses Claude Sonnet 4.6 (\$3 input / \$0.30 cache read / \$15 output per 1M tokens) for all configurations. LLM operators and BlendSQL LLM ingredients use Claude Haiku 4.5 (\$1 input / \$0.10 cache read / \$5 output per 1M tokens), a smaller model with lower costs. All LLM calls are routed through LiteLLM v1.81.16 to AWS Bedrock endpoints. Each query execution is allowed up to 30 steps with a 30-minute wall-clock timeout.

Systems. We compare three systems on the same 80-query evaluation split:

- **Agentic Text2SQL.** The agent iteratively refines and executes SQL over the database with no explicit LLM operators.
- **Agentic BlendSQL.** The agent iteratively refines and executes hybrid queries through BlendSQL engine (v0.1.14), which embeds LLM operators as inline ingredients within SQL rather than exposing them as separate tool calls.
- **Agentic Query Execution (ours).** The agent receives the full MCP tool suite with separate semantic operators. We report results both without and with EnumGRPO; the latter augments the agent with an experience pool of planning heuristics distilled from learning-stage rollouts.

EnumGRPO settings. Learning runs on the 40-query learning split. Each batch contains 2 queries with a group rollout size of 5, yielding 10 concurrent executions per batch. The plan enumerator seeds each rollout with a distinct axis-value assignment from the plan space (Section 3.1). The total learning thus requires 200 rollouts across 40 learning queries, completed in roughly 8.4 hours wall-clock time. The composite reward weights are $w_{\text{EX}} = 0.5$, $w_{\text{TUPLE}} = 0.3$, $w_{\text{CELL}} = 0.2$, with cost penalty weight $w_{\text{COST}} = 0.2$ (split as $w_{\text{TOKEN}} = 0.7$, $w_{\text{STEP}} = 0.3$).

5 Experimental Results

We now evaluate our system on the 80-query evaluation split of SWAN. Our analysis addresses three questions: (1) Does Agentic Query Execution with explicit semantic operators outperform hybrid query approaches? (2) Does EnumGRPO improve the quality-cost frontier beyond the base system? (3) Where do efficiency gains originate?

5.1 Main Results

Table 3 reports per-database answer quality alongside workflow efficiency and token costs across all five configurations, averaged over 3 independent runs.

Answer quality. Agentic Query Execution with EnumGRPO achieves the highest execution accuracy at 35.4%, an 84% relative improvement over Agentic Text2SQL (19.2%) and an 18% relative gain over Agentic BlendSQL (30.0%). The fine-grained metrics show a similar pattern: Agentic Query Execution with EnumGRPO reaches a CellF1 score of 41.4%, while the base system reaches 39.9%; all three outperform Agentic BlendSQL (37.4%) overall. BlendSQL performs best on California Schools, but only with substantially higher LLM-operator usage.

The improvements from EnumGRPO (32.1% vs. 35.4% EX) shows that the experience pool provides actionable planning heuristics. The agent learns when to prefer code-driven over data-driven execution, when targeted scope suffices, and when narrow projection preserves enough context. These are precisely the axis-level decisions EnumGRPO is designed to improve.

Workflow efficiency. Agentic Text2SQL is fast at 142s per query because it does not invoke time-consuming LLM operators, but this efficiency comes with low answer quality. Among systems that do use semantic operators, Agentic Query Execution is faster than Agentic BlendSQL (235s vs. 346s) by decoupling LLM operators from hybrid SQL execution. EnumGRPO further reduces latency to 153s per query, partly by shortening the average workflow from 17.05 to 15.79 steps, indicating that the learned heuristics help the

agent settle on an operator strategy earlier instead of iterating through trial-and-error refinements.

Cost analysis. LLM-operator cost highlights a major difference between the systems. Normalized per query from the totals in Table 3, Agentic BlendSQL consumes 1.01M LLM-operator tokens and incurs \$3.42 on LLM operations alone. By contrast, Agentic Query Execution with EnumGRPO spends only \$0.011 per query on LLM operations while delivering the highest answer quality. The base configuration (\$0.039 per query) already offers an 88× LLM-operator cost reduction over BlendSQL, and EnumGRPO shaves an additional 72% by learning to select cost-efficient plan-axis configurations. This makes the optimized agentic workflow practical for repeated analytical workloads where LLM-operator cost would otherwise dominate execution.

Held-out transfer. To test whether EnumGRPO learns workflow optimization heuristics transferable across schemas, we run a held-out variant. For each evaluation database, the agent is trained only on learning queries from the other databases and never sees any queries from the target database. Under this setting, held-out EnumGRPO preserves answer quality relative to the base system (32.9% vs. 32.1% EX) while reducing average latency from 235s to 110s and LLM-operator cost from \$3.1120 to \$0.5491. The experience pool therefore carries cross-schema signal: the agent transfers operator-selection and scope-control heuristics learned on other databases to an unseen target schema.

Compared with the main EnumGRPO configuration, the held-out variant shows more efficient workflow execution (110s vs. 153s, \$0.5491 vs. \$0.8640) but less accurate answers (32.9% vs. 35.4% EX). This tradeoff reflects a more conservative policy: without target-schema affordances, the agent follows shorter trajectories, invokes narrower semantic operators, or falls back earlier to simpler SQL or code paths. The main configuration uses both target and non-target learning queries, spending additional computation on harder queries that the held-out policy abandons or simplifies.

5.2 LLM Operator Usage Analysis

The aggregate cost numbers above come from two trace-level effects: how often a workflow invokes LLM operators and how much data each invocation sends to the model.

First, *fewer queries require LLM operators* under EnumGRPO (40/80 vs. 51/80 for the base configuration). This indicates that the learned heuristics help the agent recognize when the semantic gap can be handled by SQL-visible evidence or code-driven rules, avoiding unnecessary LLM-operator invocations. Second, *when LLM operators are invoked, they consume far fewer tokens*. The base system averages 52K tokens per LLM-operator-using query, while EnumGRPO reduces this to 18K, a 64% reduction. This reflects the optimizer learning to prefer targeted selectivity scope and narrow projection width, reducing the number of rows and columns passed to the LLM. Third, *output tokens drop dramatically* from the base system (2,301) to EnumGRPO (775), consistent with more frequent use of aggregate operators or code-driven synthesis rather than verbose per-row scalar processing.

This mechanism also explains the gap to BlendSQL. Although BlendSQL already includes a query optimizer that pushes SQL predicates before LLM ingredients when possible [38], each ingredient

Table 3: Per-database evaluation on the 80-query SWAN split, averaged over 3 runs. LLM-op token counts and costs are totals across all queries in each database. Held-out EnumGRPO uses learning queries only from non-target databases. The Overall block reports aggregate results over the full 80-query split. Values denote mean $\pm$ sd. Best mean per row is bolded.

	Agentic Text2SQL	Agentic BlendSQL	Agentic DB Analytics	Agentic DB Analytics w/ EnumGRPO (Held-out)	Agentic DB Analytics w/ EnumGRPO
<i>California Schools</i> (19 queries)					
Execution Accuracy (%)	36.8 $\pm$ 0.0	49.1 $\pm$ 6.1	40.4 $\pm$ 3.0	38.6 $\pm$ 3.0	47.4 $\pm$ 9.1
Tuple-level F1 (%)	36.8 $\pm$ 0.0	50.2 $\pm$ 6.8	40.5 $\pm$ 2.9	38.6 $\pm$ 3.0	47.4 $\pm$ 9.1
Cell-level F1 (%)	45.7 $\pm$ 4.3	54.1 $\pm$ 6.9	44.7 $\pm$ 0.5	43.3 $\pm$ 2.7	51.5 $\pm$ 8.2
Avg Walltime (s)	125 $\pm$ 7	281 $\pm$ 72	202 $\pm$ 50	112 $\pm$ 2	118 $\pm$ 15
LLM-op Token Usage	–	13,413,563 $\pm$ 4,641,790	7,980 $\pm$ 4,755	10,938 $\pm$ 7,039	1,640 $\pm$ 1,861
Input	–	5,263,532 $\pm$ 1,706,721	7,435 $\pm$ 4,470	10,089 $\pm$ 6,568	1,545 $\pm$ 1,775
Output	–	8,150,031 $\pm$ 2,940,827	538 $\pm$ 275	842 $\pm$ 686	89 $\pm$ 77
LLM-op Total Cost (\$)	–	46.0142 $\pm$ 16.4023	0.0108 $\pm$ 0.0055	0.0025 $\pm$ 0.0029	0.0019 $\pm$ 0.0019
<i>European Football</i> (20 queries)					
Execution Accuracy (%)	13.3 $\pm$ 2.9	18.3 $\pm$ 7.6	26.7 $\pm$ 2.9	26.7 $\pm$ 5.8	28.3 $\pm$ 2.9
Tuple-level F1 (%)	16.1 $\pm$ 4.4	21.9 $\pm$ 6.1	28.6 $\pm$ 2.6	29.7 $\pm$ 3.2	29.1 $\pm$ 2.0
Cell-level F1 (%)	16.1 $\pm$ 4.4	21.9 $\pm$ 6.1	28.6 $\pm$ 2.6	29.7 $\pm$ 3.2	29.1 $\pm$ 2.0
Avg Walltime (s)	153 $\pm$ 13	539 $\pm$ 116	287 $\pm$ 88	97 $\pm$ 4	154 $\pm$ 11
LLM-op Token Usage	–	45,934,740 $\pm$ 16,653,298	1,541,533 $\pm$ 37,218	28,940 $\pm$ 17,610	102,473 $\pm$ 85,546
Input	–	18,242,440 $\pm$ 6,730,969	1,464,827 $\pm$ 27,794	26,587 $\pm$ 17,481	97,867 $\pm$ 82,779
Output	–	27,692,300 $\pm$ 9,923,941	76,720 $\pm$ 18,556	2,353 $\pm$ 129	4,607 $\pm$ 2,789
LLM-op Total Cost (\$)	–	156.7047 $\pm$ 56.3482	1.8480 $\pm$ 0.1030	0.0153 $\pm$ 0.0076	0.1207 $\pm$ 0.0965
<i>Formula One</i> (22 queries)					
Execution Accuracy (%)	19.7 $\pm$ 2.6	31.8 $\pm$ 4.5	36.4 $\pm$ 4.5	40.9 $\pm$ 4.5	37.9 $\pm$ 11.4
Tuple-level F1 (%)	27.7 $\pm$ 2.9	40.9 $\pm$ 4.5	47.9 $\pm$ 7.1	48.5 $\pm$ 3.3	44.9 $\pm$ 11.4
Cell-level F1 (%)	38.8 $\pm$ 1.3	47.2 $\pm$ 2.5	55.9 $\pm$ 5.6	56.4 $\pm$ 3.8	52.8 $\pm$ 10.1
Avg Walltime (s)	107 $\pm$ 5	176 $\pm$ 24	154 $\pm$ 44	76 $\pm$ 4	99 $\pm$ 4
LLM-op Token Usage	–	3,477,569 $\pm$ 237,049	24,765 $\pm$ 9,343	23,335 $\pm$ 6,510	12,973 $\pm$ 5,666
Input	–	1,443,325 $\pm$ 105,944	22,953 $\pm$ 9,078	20,878 $\pm$ 5,716	11,073 $\pm$ 5,521
Output	–	2,034,237 $\pm$ 152,919	1,811 $\pm$ 406	2,449 $\pm$ 798	1,899 $\pm$ 409
LLM-op Total Cost (\$)	–	11.6145 $\pm$ 0.8391	0.0315 $\pm$ 0.0113	0.0220 $\pm$ 0.0066	0.0205 $\pm$ 0.0067
<i>Superhero</i> (19 queries)					
Execution Accuracy (%)	7.0 $\pm$ 3.0	21.1 $\pm$ 0.0	24.6 $\pm$ 3.0	24.6 $\pm$ 3.0	28.1 $\pm$ 6.1
Tuple-level F1 (%)	10.4 $\pm$ 1.3	25.2 $\pm$ 1.9	27.6 $\pm$ 0.3	25.4 $\pm$ 3.2	30.5 $\pm$ 7.4
Cell-level F1 (%)	10.4 $\pm$ 1.3	25.9 $\pm$ 0.9	28.3 $\pm$ 1.0	25.4 $\pm$ 3.2	31.2 $\pm$ 8.3
Avg Walltime (s)	187 $\pm$ 25	403 $\pm$ 74	306 $\pm$ 68	160 $\pm$ 14	251 $\pm$ 31
LLM-op Token Usage	–	18,323,695 $\pm$ 2,294,966	1,072,062 $\pm$ 182,925	593,364 $\pm$ 215,010	622,503 $\pm$ 188,289
Input	–	7,999,595 $\pm$ 931,915	1,033,765 $\pm$ 179,460	570,640 $\pm$ 209,405	598,095 $\pm$ 180,174
Output	–	10,324,106 $\pm$ 1,363,114	38,298 $\pm$ 3,486	22,718 $\pm$ 5,641	24,409 $\pm$ 8,205
LLM-op Total Cost (\$)	–	59.6201 $\pm$ 7.7482	1.2249 $\pm$ 0.1964	0.5098 $\pm$ 0.1592	0.7201 $\pm$ 0.2211
Overall (80 queries)					
Execution Accuracy (%)	19.2 $\pm$ 1.4	30.0 $\pm$ 0.0	32.1 $\pm$ 1.9	32.9 $\pm$ 2.2	35.4 $\pm$ 0.7
Tuple-level F1 (%)	22.8 $\pm$ 2.2	34.6 $\pm$ 0.9	36.5 $\pm$ 2.1	36.0 $\pm$ 1.6	38.1 $\pm$ 0.8
Cell-level F1 (%)	28.0 $\pm$ 2.2	37.4 $\pm$ 0.9	39.9 $\pm$ 1.3	39.2 $\pm$ 1.6	41.4 $\pm$ 1.2
Avg Walltime (s)	142 $\pm$ 12	346 $\pm$ 53	235 $\pm$ 62	110 $\pm$ 4	153 $\pm$ 14
LLM-op Token Usage	–	81,149,547 $\pm$ 10,362,371	2,646,347 $\pm$ 206,756	656,576 $\pm$ 215,943	739,600 $\pm$ 266,288
Input	–	32,948,880 $\pm$ 4,364,556	2,528,987 $\pm$ 186,197	628,193 $\pm$ 210,313	708,587 $\pm$ 255,544
Output	–	48,200,667 $\pm$ 6,002,093	117,360 $\pm$ 21,425	28,363 $\pm$ 5,740	31,013 $\pm$ 10,832
LLM-op Total Cost (\$)	–	273.9520 $\pm$ 34.3680	3.1120 $\pm$ 0.2880	0.5491 $\pm$ 0.1600	0.8640 $\pm$ 0.3120

is still evaluated at the position specified by the hybrid query. In SWAN, this causes BlendSQL to process large intermediate relations through the LLM, averaging 1.01M LLM-operator tokens per query. Agentic Query Execution decouples these choices, giving the optimizer explicit control over when LLM operators run and how much data they receive.

5.3 Scalability Analysis

To test how latency and cost respond to data volume, we synthesize scaled copies of each database at 0.25 $\times$, 0.5 $\times$, 2 $\times$, and 4 $\times$ the primary entity table size and rerun the evaluation at each scale. Scaling uses one primary entity table per database as the cardinality

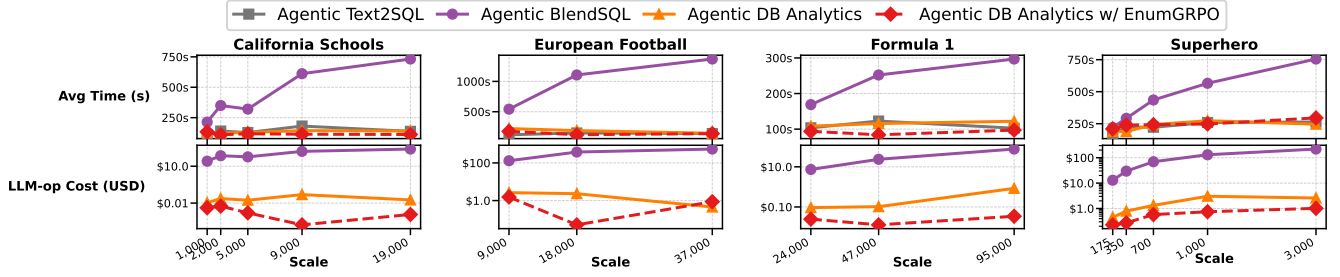


Figure 3: Scalability of average per-query latency (up) and total LLM-operator cost (down) as the primary table size in each database increases. To preserve the answer-bearing rows, European Football and Formula 1 cannot be downscaled below their original primary entity table sizes. Overall, Agentic BlendSQL’s latency and cost increase with scale, whereas both Agentic Query Execution configurations remain nearly flat by invoking LLM operators selectively on scoped intermediate results.

Table 4: Distribution of the 32 learned experiences across the five plan-space axes, with one example experience per axis quoted verbatim from the final pool ε . The *Polarity* column shows how many experiences favor each axis value.

Axis	#	%	Polarity	Example Experience
Execution paradigm	9	28%	code 7 : data 2	“When a join returns 0 rows or fewer than expected, immediately switch to an alternative SQL resolution rather than retrying variations.”
Selectivity scope	8	25%	targeted 5 : full 3	“Run describe_relation once on all relevant tables, then immediately attempt the analytical SQL; never re-examine schema with multiple calls.”
Operator placement	6	19%	pre 3 : post 3	“Always SQL-filter to the smallest candidate set before any LLM call; exhaust all structural join/GROUP BY paths first.”
Projection width	6	19%	narrow 3 : wide 3	“Use MIN/MAX/AVG on ambiguous numeric columns to infer true meaning before misidentifying the column or invoking any LLM operator.”
Operator type	3	9%	reduce 3 : map 0	“Prefer llm_reduce (one batch call) over llm_map (per-row); 5–11× cheaper with comparable accuracy.”

knob, with non-primary tables left intact to preserve join validity. When downscaling, rows needed to preserve gold-answer values are protected before subsampling. Removing them would collapse the semantic gap that necessitates LLM inference, making queries purely relational or unanswerable.

The results are shown in Figure 3. Agentic BlendSQL diverges from the other systems as scale increases, most clearly on Superhero: from 0.25× to 4×, its average wall time rises from 220s to 755s and its total LLM-operator cost rises from \$11.4 to \$164.9. This growth occurs because inline LLM ingredients process rows in intermediate relations whose cardinalities increase with data size. By contrast, Agentic Text2SQL and both Agentic Query Execution configurations remain nearly flat on both metrics, confirming that decoupling LLM operators from the query execution path is essential for predictable scaling.

A subtler pattern appears for Agentic Query Execution at larger scales. On California Schools and European Football, LLM-operator cost decreases rather than grows. The agent increasingly selects SQL-first or highly selective execution paths, such as pure SQL fallbacks or aggressive WHERE IN pre-filters. This behavior shows that agentic execution can adaptively bound semantic-operator cost as data size increases. It also highlights a tradeoff: overly conservative routing may skip useful semantic evidence. EnumGRPO shows the same tendency while keeping costs lower overall (Superhero 0.25×:

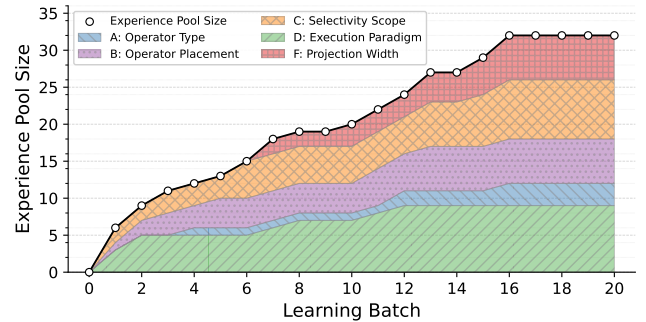


Figure 4: Growth of the experience pool ε over learning batches by database. The pool accumulates rapidly in early batches and converges at later stages of learning.

\$0.18 to 4×: \$0.81). Future optimizers could improve scale robustness by conditioning learned experiences on relation cardinality and expected semantic coverage.

5.4 Analysis of the Learned Experience Pool

To understand what the optimizer learns, we examine how the experience pool grows during learning (Figure 4) and then categorize its

final contents by plan-space axis. The pool accumulates experiences quickly in early batches and plateaus at 32 around batch 16, suggesting that later rollouts mostly reinforce existing heuristics rather than introduce new ones. Execution-paradigm experiences appear first and remain the largest category throughout learning, consistent with code-driven vs. data-driven execution being the dominant cost lever. Projection-width experiences emerge later, suggesting that context-width decisions become useful after the optimizer has already learned broader execution and scoping heuristics.

We manually categorize the 32 experiences in the final pool ϵ by the plan-space axis they address most directly. Table 4 reports the distribution across the five planning axes, together with one representative experience quoted verbatim from the pool. Each experience is assigned to exactly one category based on the primary structural decision it targets.

The final pool is dominated by *execution paradigm* and *selectivity scope*, which together account for over half of all entries. This concentration matches the steepest part of the quality-cost trade-off: execution paradigm determines whether LLM usage is $O(1)$ or $O(|R|)$, while selectivity scope controls how many rows reach those calls. By contrast, the small number of *operator type* experiences suggests that scalar-vs.-aggregate choices are often subsumed by execution paradigm, since code-driven workflows avoid per-row LLM operators entirely. The remaining decisions are more conditional: *operator placement* and *projection width* are balanced across the polarities, indicating that the optimizer learns situational rules rather than blanket preferences.

5.5 Efficiency-Quality Tradeoff

The 8.4 hours of EnumGRPO learning (200 rollouts over 40 queries) produce a fixed experience pool that requires no additional computation at evaluation time beyond the extra prompt tokens. The amortized learning cost is modest relative to the per-query LLM-operator savings achieved during deployment: across 80 evaluation queries, EnumGRPO saves approximately \$2.25 in LLM-operator cost compared to the base configuration, while the entire learning costs roughly \$41.2 in API calls (200 rollouts $\times$ \$0.206 average cost per rollout). The optimizer thus pays for itself within approximately 1,460 queries at the observed LLM-operator savings rate.

6 Related Work

6.1 Semantic Query Processing

Recent systems have shown how to integrate semantic inference into data processing engines. BlendSQL extends SQL with LLM-backed operators such as LLMap and LLMQA for hybrid question answering [10]. Zhao *et al.* study hybrid querying over relational databases and LLMs, introducing the SWAN benchmark and the HQDL baseline based on schema expansion and UDF-style calls [38]. CAESURA treats language models as planners that compile natural-language requests into multi-modal execution pipelines [35], while *nsDB* argues for a neuro-symbolic DBMS with joint latency-accuracy objectives [37]. Palimpsest goes further by casting AI-powered analytics as declarative optimization over model, prompt, and execution choices along a Pareto frontier [21], and Weaver dynamically interleaves SQL and LLM reasoning for table question answering [12]. These engines establish the importance of

hybrid symbolic-neural analytics, but they largely optimize a fixed query, declarative program, or generated plan rather than deciding online from intermediate results which operator to apply next.

6.2 Semantic Operator Optimization

Another line of work focuses on optimizing the expensive semantic operators themselves. LOTUS introduces semantic operators with statistical accuracy guarantees and optimization strategies for filtering, joining, grouping, and top- k -style workloads [29]. iPDB extends SQL with ML and LLM predicates and optimizes semantic queries via predicate ordering, prompt deduplication, and multi-row marshaling [15]. Outside the database engine proper, Liu *et al.* reorder LLM requests in relational workloads to improve cache locality and serving efficiency [22]. Classical work on efficient execution of UDFs [8] also provides a useful structural analogue, since it likewise shows that operator ordering and implementation details can dominate runtime. Our setting differs because the optimization target is not a single semantic operator or a fixed semantic SQL plan, but an execution-time workflow whose SQL and LLM steps, placement, and granularity are chosen jointly as execution unfolds.

6.3 Learned Query Optimization

Adaptive query optimization and learned query optimizers provide the database lineage for our approach. Eddies pioneered continuously adaptive query processing [3]. Bao learns per-query hint policies on top of an existing optimizer [23], Balsa learns an optimizer without expert demonstrations via reinforcement learning [36], and Lero uses learning-to-rank to improve plan selection in existing DBMSs [39]. SkinnerDB is especially relevant because it learns join orders during execution and frames the problem through regret-bounded processing [34]. Learned cost models further inform this space by improving plan assessment for complex workloads [20]. Our work extends this adaptive optimization perspective from conventional relational operators to heterogeneous SQL and semantic operators, where both execution cost and answer quality are data-dependent and may only become clear after intermediate results are observed.

7 Conclusion

We formulated agent workflow optimization, a runtime query optimization problem for the new paradigm of Agentic Query Execution, where planning is interleaved with execution over intermediate results. The main lever is not simply adding semantic operators to SQL, but optimizing when they run and how much data they see. We instantiated this in EnumGRPO: it enumerates candidate workflows along these axes and distills their quality-cost feedback into reusable heuristics, reaching a new state of the art on SWAN at 35.4% execution accuracy and \$0.011 per query.

Open directions include consolidating the experience pool and retrieving by relevance, so the policy draws on the experiences most pertinent to each query. Continual learning over incoming queries could further adapt the policy post-deployment to shifting data and workloads. More fundamentally, as database systems incorporate approximate and expensive semantic operators, optimization must move beyond choosing a static optimal plan and toward learning policies that adapt to uncertainty, evidence, and cost at runtime.

References

- [1] Daniel J. Abadi, Samuel Madden, and Nabil Hachem. 2008. Column-stores vs. row-stores: how different are they really?. In *Proceedings of the ACM SIGMOD International Conference on Management of Data, SIGMOD 2008, Vancouver, BC, Canada, June 10-12, 2008*, Jason Tsong-Li Wang (Ed.). ACM, 967–980. <https://doi.org/10.1145/1376616.1376712>
- [2] Lakshya A. Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziemis, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J. Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Daniel Klein, Matei Zaharia, and Omar Khattab. 2025. GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning. *CoRR abs/2507.19457* (2025). <https://doi.org/10.48550/ARXIV.2507.19457> arXiv:2507.19457
- [3] Ron Avnur and Joseph M. Hellerstein. 2000. Eddies: Continuously Adaptive Query Processing. In *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data, May 16-18, 2000, Dallas, Texas, USA*, Weidong Chen, Jeffrey F. Naughton, and Philip A. Bernstein (Eds.). ACM, 261–272. <https://doi.org/10.1145/342009.335420>
- [4] Yuzheng Cai, Siqi Cai, Yuchen Shi, Zihan Xu, Lichao Chen, Yulei Qin, Xiaoyu Tan, Gang Li, Zongyi Li, Haojia Lin, Yong Mao, Ke Li, and Xing Sun. 2025. Training-Free Group Relative Policy Optimization. *CoRR abs/2510.08191* (2025). <https://doi.org/10.48550/ARXIV.2510.08191> arXiv:2510.08191
- [5] Surajit Chaudhuri. 1998. An Overview of Query Optimization in Relational Systems. In *Proceedings of the Seventeenth ACM SIGACT-SIGMOD-SIGART Symposium on Principles of Database Systems, June 1-3, 1998, Seattle, Washington, USA*, Alberto O. Mendelzon and Jan Paredaens (Eds.). ACM Press, 34–43. <https://doi.org/10.1145/275487.275492>
- [6] Benoit Dherin, Michael Munn, Hanna Mazzawi, Michael Wunder, and Javier Gonzalo. 2025. Learning without training: The implicit dynamics of in-context learning. *CoRR abs/2507.16003* (2025). <https://doi.org/10.48550/ARXIV.2507.16003> arXiv:2507.16003
- [7] Mohamed Y. Eltabakh, Zan Ahmad Naeem, Mohammad Shahmeer Ahmad, Mourad Ouzzani, and Nan Tang. 2024. RetClean: Retrieval-Based Tabular Data Cleaning Using LLMs and Data Lakes. *Proc. VLDB Endow.* 17, 12 (2024), 4421–4424. <https://doi.org/10.14778/3685800.3685890>
- [8] Yannis Foufoulas and Alkis Simitsis. 2023. Efficient Execution of User-Defined Functions in SQL Queries. *Proc. VLDB Endow.* 16, 12 (2023), 3874–3877. <https://doi.org/10.14778/3611540.3611574>
- [9] Dawei Gao, Haibin Wang, Yaliang Li, Xiuyu Sun, Yichen Qian, Bolin Ding, and Jingren Zhou. 2024. Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation. *Proc. VLDB Endow.* 17, 5 (2024), 1132–1145. <https://doi.org/10.14778/3641204.3641221>
- [10] Parker Glenn, Parag Dakle, Liang Wang, and Preethi Raghavan. 2024. BlendSQL: A Scalable Dialect for Unifying Hybrid Question Answering in Relational Algebra. In *Findings of the Association for Computational Linguistics, ACL 2024, Bangkok, Thailand and virtual meeting, August 11-16, 2024 (Findings of ACL)*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, 453–466. <https://doi.org/10.18653/V1/2024.FINDINGS-ACL.25>
- [11] Goetz Graefe. 1995. The Cascades Framework for Query Optimization. *IEEE Data Eng. Bull.* 18, 3 (1995), 19–29. <http://sites.computer.org/debull/95SEP-CD.pdf>
- [12] Rohit Khoja, Devanshu Gupta, Yanjie Fu, Dan Roth, and Vivek Gupta. 2025. Weaver: Interweaving SQL and LLM for Table Reasoning. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, EMNLP 2025, Suzhou, China, November 4-9, 2025*, Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng (Eds.). Association for Computational Linguistics, 28282–28308. <https://doi.org/10.18653/V1/2025.EMNLP-MAIN.1436>
- [13] Hyeonji Kim, Byeong-Hoon So, Wook-Shin Han, and Hongrae Lee. 2020. Natural language to SQL: Where are we today? *Proc. VLDB Endow.* 13, 10 (2020), 1737–1750. <https://doi.org/10.14778/3401960.3401970>
- [14] Hanna Köpcke, Andreas Thor, and Erhard Rahm. 2010. Evaluation of entity resolution approaches on real-world match problems. *Proc. VLDB Endow.* 3, 1 (2010), 484–493. <https://doi.org/10.14778/1920841.1920904>
- [15] Udesch Kumarasinghe, Tyler Liu, Chunwei Liu, and Walid G. Aref. 2026. iPDB – Optimizing SQL Queries with ML and LLM Predicates. *CoRR abs/2601.16432* (2026). <https://doi.org/10.48550/ARXIV.2601.16432> arXiv:2601.16432
- [16] Jiale Lao, Andreas Zimmerer, Olga Ovcharenko, Tianji Cong, Matthew Russo, Gerardo Vitagliano, Michael Cochez, Fatma Özcan, Gautam Gupta, Thibaud Hotetier, H. V. Jagadish, Kris Kissel, Sebastian Schelter, Andreas Kipf, and Immanuel Trummer. 2025. SemBench: A Benchmark for Semantic Query Processing Engines. *CoRR abs/2511.01716* (2025). <https://doi.org/10.48550/ARXIV.2511.01716> arXiv:2511.01716
- [17] Boyan Li, Yuyu Luo, Chengliang Chai, Guoliang Li, and Nan Tang. 2024. The Dawn of Natural Language to SQL: Are We Fully Ready? [Experiment, Analysis & Benchmark]. *Proc. VLDB Endow.* 17, 11 (2024), 3318–3331. <https://doi.org/10.14778/3681954.3682003>
- [18] Jinyang Li, Binyuan Hui, Ge Qu, Jiaxi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, et al. 2024. Can LLM Already Serve as a Database Interface? A Blg Bench for Large-Scale Database Grounded Text-to-SQLs. *Advances in Neural Information Processing Systems* 36 (2024).
- [19] Yuliang Li, Jinfeng Li, Yoshihiko Suhara, AnHai Doan, and Wang-Chiew Tan. 2020. Deep Entity Matching with Pre-Trained Language Models. *Proc. VLDB Endow.* 14, 1 (2020), 50–60. <https://doi.org/10.14778/3421424.3421431>
- [20] Yan Li, Liwei Wang, Sheng Wang, Yuan Sun, Bolong Zheng, and Zhiyong Peng. 2024. A learned cost model for big data query processing. *Inf. Sci.* 670 (2024), 120650. <https://doi.org/10.1016/j.ins.2024.120650>
- [21] Chunwei Liu, Matthew Russo, Michael J. Cafarella, Lei Cao, Peter Baile Chen, Zui Chen, Michael J. Franklin, Tim Kraska, Samuel Madden, Rana Shahout, and Gerardo Vitagliano. 2025. Palimpsest: Optimizing AI-Powered Analytics with Declarative Query Processing. In *15th Conference on Innovative Data Systems Research, CIDR 2025, Amsterdam, The Netherlands, January 19-22, 2025*. <https://cidrdb.org>. <https://vldb.org/cidrdb/2025/palimpsest-optimizing-ai-powered-analytics-with-declarative-query-processing.html>
- [22] Shu Liu, Asim Biswal, Audrey Cheng, Xiangxi Mo, Shiyi Cao, Joseph E. Gonzalez, Ion Stoica, and Matei Zaharia. 2024. Optimizing LLM Queries in Relational Workloads. *CoRR abs/2403.05821* (2024). <https://doi.org/10.48550/ARXIV.2403.05821> arXiv:2403.05821
- [23] Ryan Marcus, Parimarjan Negi, Hongzi Mao, Nesime Tatbul, Mohammad Alizadeh, and Tim Kraska. 2021. Bao: Making Learned Query Optimization Practical. In *Proceedings of the 2021 International Conference on Management of Data, SIGMOD 2021, Virtual Event, China, June 20-25, 2021*. ACM, 1275–1288. <https://doi.org/10.1145/3448016.3452838>
- [24] David Menestrina, Steven Whang, and Hector Garcia-Molina. 2010. Evaluating Entity Resolution Results. *Proc. VLDB Endow.* 3, 1 (2010), 208–219. <https://doi.org/10.14778/1920841.1920871>
- [25] Prashanth Menon, Andrew Pavlo, and Todd C. Mowry. 2017. Relaxed Operator Fusion for In-Memory Databases: Making Compilation, Vectorization, and Prefetching Work Together At Last. *Proc. VLDB Endow.* 11, 1 (2017), 1–13. <https://doi.org/10.14778/3151113.3151114>
- [26] Guido Moerkotte and Thomas Neumann. 2006. Analysis of Two Existing and One New Dynamic Programming Algorithm for the Generation of Optimal Bushy Join Trees without Cross Products. In *Proceedings of the 32nd International Conference on Very Large Data Bases, Seoul, Korea, September 12-15, 2006*, Umeshwar Dayal, Kyu-Young Whang, David B. Lomet, Gustavo Alonso, Guy M. Lohman, Martin L. Kersten, Sang Kyun Cha, and Young-Kuk Kim (Eds.). ACM, 930–941. <http://dl.acm.org/citation.cfm?id=1164207>
- [27] Thomas Neumann. 2011. Efficiently Compiling Efficient Query Plans for Modern Hardware. *Proc. VLDB Endow.* 4, 9 (2011), 539–550. <https://doi.org/10.14778/2002938.2002940>
- [28] Liana Patel, Siddharth Jha, Carlos Guestrin, and Matei Zaharia. 2024. LOTUS: Enabling Semantic Queries with LLMs Over Tables of Unstructured and Structured Data. *CoRR abs/2407.11418* (2024). <https://doi.org/10.48550/ARXIV.2407.11418> arXiv:2407.11418
- [29] Liana Patel, Siddharth Jha, Melissa Pan, Harshit Gupta, Parth Asawa, Carlos Guestrin, and Matei Zaharia. 2025. Semantic Operators and Their Optimization: Enabling LLM-Based Data Processing with Accuracy Guarantees in LOTUS. *Proc. VLDB Endow.* 18, 11 (2025), 4171–4184. <https://doi.org/10.14778/3749646.3749685>
- [30] Patricia G. Selinger, Morton M. Astrahan, Donald D. Chamberlin, Raymond A. Lorie, and Thomas G. Price. 1979. Access Path Selection in a Relational Database Management System. In *Proceedings of the 1979 ACM SIGMOD International Conference on Management of Data, Boston, Massachusetts, USA, May 30 - June 1, Philip A. Bernstein (Ed.)*. ACM, 23–34. <https://doi.org/10.1145/582095.582099>
- [31] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. 2024. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. *CoRR abs/2402.03300* (2024). <https://doi.org/10.48550/ARXIV.2402.03300> arXiv:2402.03300
- [32] Kefan Song, Amir Moeini, Peng Wang, Lei Gong, Rohan Chandra, Yanjun Qi, and Shangdong Zhang. 2026. Reward Is Enough: LLMs Are In-Context Reinforcement Learners. In *Proceedings of the Fourteenth International Conference on Learning Representations, ICLR 2026*. arXiv:2506.06303
- [33] Sandeep Tata, Navneet Potti, James B. Wendt, Lauro Beltrão Costa, Marc Najork, and Beliz Gunel. 2021. Glean: Structured Extractions from Templatic Documents. *Proc. VLDB Endow.* 14, 6 (2021), 997–1005. <https://doi.org/10.14778/3447689.3447703>
- [34] Immanuel Trummer, Junxiong Wang, Ziyun Wei, Deepak Maram, Samuel Moseley, Saehan Jo, Joseph Antonakakis, and Ankush Rayabari. 2021. SkinnerDB: Regret-bounded Query Evaluation via Reinforcement Learning. *ACM Trans. Database Syst.* 46, 3 (2021), 1–45. <https://doi.org/10.1145/3464389>
- [35] Matthias Urban and Carsten Binnig. 2024. CAESURA: Language Models as Multi-Modal Query Planners. In *14th Conference on Innovative Data Systems Research, CIDR 2024, Chaminade, CA, USA, January 14-17, 2024*. <https://www.cidrdb.org/cidr2024/papers/p14-urban.pdf>
- [36] Zongheng Yang, Wei-Lin Chiang, Sifei Luan, Gautam Mittal, Michael Luo, and Ion Stoica. 2022. Balsa: Learning a Query Optimizer Without Expert Demonstrations. In *Proceedings of the 2022 International Conference on Management of Data, SIGMOD 2022, Philadelphia, PA, USA, June 12-17, 2022*. ACM, 931–944.

- <https://doi.org/10.1145/3514221.3517885>
- [37] Ye Yuan, Bo Tang, Tianfei Zhou, Zhiwei Zhang, and Jianbin Qin. 2024. nsDB: Architecting the Next Generation Database by Integrating Neural and Symbolic Systems (Vision). *Proc. VLDB Endow.* 17, 11 (2024), 3283–3289. <https://doi.org/10.14778/3681954.3682000>
- [38] Fuheng Zhao, Divyakant Agrawal, and Amr El Abbadi. 2024. Hybrid Querying Over Relational Databases and Large Language Models. *CoRR* abs/2408.00884 (2024). <https://doi.org/10.48550/ARXIV.2408.00884> arXiv:2408.00884
- [39] Rong Zhu, Wei Chen, Bolin Ding, Xingguang Chen, Andreas Pfadler, Ziniu Wu, and Jingren Zhou. 2023. Lero: A Learning-to-Rank Query Optimizer. *Proc. VLDB Endow.* 16, 6 (2023), 1466–1479. <https://doi.org/10.14778/3583140.3583160>